

Sidi Bel Abbès on 09/ 01/2025

FILE STRUCTURE – Final Exam

Time: 2h:30m

Questions

- Why is it important to distinguish between storing data and storing programs in files? Illustrate with examples how these two uses differ.
- Discuss the role of older technologies, such as tape drives, in the context of data storage evolution,
- Compare the relationship between file organization and access methods.
- What is the primary advantage of creating a second index?
- What does it mean for a B-Tree to have branches of the same length, and why is this important?

Exercise 1:

A file organization uses **indexes** to locate files on a workspace. These indexes must be unique and fall within a valid range. Common problems include:

1. **Duplicate Indexes:** Two or more files share the same index.
2. **Missing Indexes:** Some files lack an assigned index.
3. **Invalid Indexes:** An index is outside the valid range.

You are given a list of files, with the following structure:

```
Files = [  
  {"name": "file1.txt", "index": 3},  
  {"name": "file2.txt", "index": 5},  
  {"name": "file3.txt", "index": 3}, # Duplicate index  
  {"name": "file4.txt", "index": -1}, # Invalid index  
  {"name": "file5.txt", "index": 12}, # Invalid index  
  {"name": "file6.txt", "index": None}, # Missing index ]
```


- Write an pseudo algorithm that:
 - Identifies files with duplicate indexes.
 - Identifies files with missing or invalid indexes.
 - Corrects these issues by assigning a unique, valid index to each file. The new index should come from the range [0, 9] and must not already be used.
- Display the file list before and after correction in a table format.
- How can you efficiently simulate and process a file organization with millions of files to detect and resolve index-related issues while optimizing memory usage and performance?

Expected Output

Before correction:

Name	Index
-----	-----
file1.txt	3
file2.txt	5
file3.txt	3 (duplicate)
file4.txt	-1 (invalid)
file5.txt	12 (invalid)
file6.txt	None (missing)

After correction:

Name	Index
-----	-----
file1.txt	3
file2.txt	5
file3.txt	6
file4.txt	7
file5.txt	8
file6.txt	9

Exercise 2:

Create a **B-tree of order N=3** (i.e., each node can contain at most 2 keys).

- Insert the following keys sequentially into the initially empty B-tree: [15,5,10,20,25,30].
- Ensure that the **B-tree properties are maintained**:
 - Nodes (except the root) must be at least half-full.
 - Nodes can have at most N-1 keys.
 - The tree is balanced, meaning all leaves are at the same depth.
- Implement the insertion algorithm with balancing in pseudocode.

Exercise 3:

You are given a hash table of size $m=8$, and you need to insert the following keys into the table: 12, 20, 28, 36, 44 Hash Functions: Primary hash function: $h_1(k) = k \bmod m$ Secondary hash function: $h_2(k) = 2 + (k \bmod 4)$, For each key k , the double hash function is given by:

$$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

- Insert the keys into the hash table using **double hashing**.
- Identify any **cycles** during the insertion process and explain why they occur.
- Propose a solution to avoid cycles and successfully insert all keys.

Good luck

- Write an pseudo algorithm that:
 - Identifies files with duplicate indexes.
 - Identifies files with missing or invalid indexes.
 - Corrects these issues by assigning a unique, valid index to each file. The new index should come from the range $[0, 9]$ and must not already be used.
- Display the file list before and after correction in a table format.
- How can you efficiently simulate and process a file organization with millions of files to detect and resolve index-related issues while optimizing memory usage and performance?

Expected Output

Before correction:

Name	Index
file1.txt	3
file2.txt	5
file3.txt	3 (duplicate)
file4.txt	-1 (invalid)
file5.txt	12 (invalid)
file6.txt	None (missing)

After correction:

Name	Index
file1.txt	3
file2.txt	5
file3.txt	6
file4.txt	7
file5.txt	8
file6.txt	9

Exercise 2:

Create a **B-tree of order $N=3$** (i.e., each node can contain at most 2 keys).

- Insert the following keys sequentially into the initially empty B-tree: **[15,5,10,20,25,30]**.
- Ensure that the **B-tree properties are maintained**:
 - Nodes (except the root) must be at least half-full.
 - Nodes can have at most $N-1$ keys.
 - The tree is balanced, meaning all leaves are at the same depth.
- Implement the insertion algorithm with balancing in pseudocode.

Exercise 3:

You are given a hash table of size $m=8$, and you need to insert the following keys into the table: **12, 20, 28, 36, 44** Hash Functions: Primary hash function: $h_1(k) = k \bmod m$ Secondary hash function: $h_2(k) = 2 + (k \bmod 4)$, For each key k , the double hash function is given by:

$$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

- Insert the keys into the hash table using **double hashing**.
- Identify any **cycles** during the insertion process and explain why they occur.
- Propose a solution to avoid cycles and successfully insert all keys.

Good luck